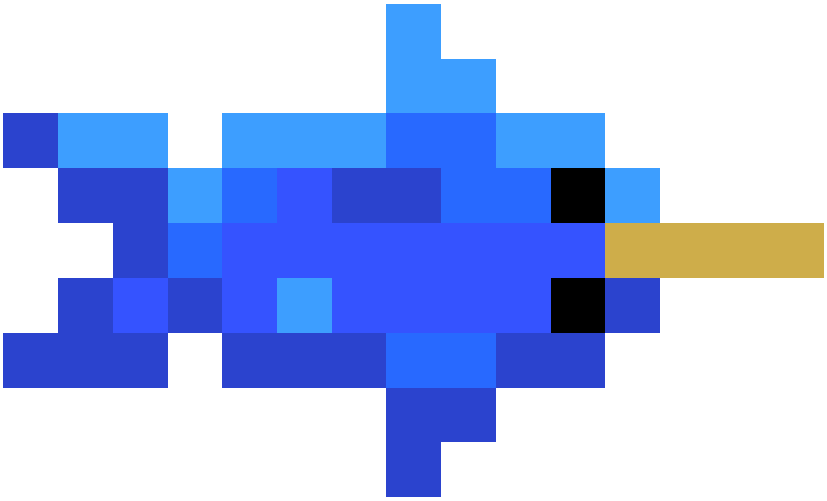
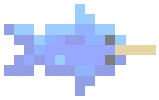
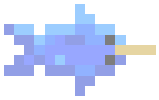


Custom Health/Loading/Progress Bar/Transition Components
General Guide

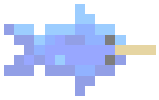


Version Notes

Date: October 2, 2023
Project: 01
Version: 1.0
By: Isaac López Procopio - Pixel Narval
in collaboration with: Chantal López Procopio



Contents



About

Author's notes

My motivation for publishing assets on the Unity Asset Store is to simplify game development. I've noticed that creating basic game elements like title screens and character controllers often takes up a lot of time. These tasks can be repetitive and keep us from focusing on other important aspects of game creation. Game engines help by providing ready-made components, but even building engines from scratch can be time-consuming.

To address this, I want to offer easily adaptable and customizable game components. This way, developers can save time and effort, which can be used for other crucial parts of their games. Overall, my goal is to make game development more efficient and enjoyable by streamlining the process of creating essential game elements.

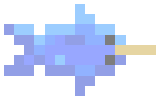
Coming next

Our goal is to produce more theme-based custom bars, loading scenes, custom menus many new useful components, some of them will be added to existing plugins, such as this one, to deliver a better product, so any feedback is well received for future development.

Contact Us

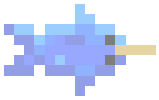
Don't hesitate to get in touch with us in case you find any bugs, if you want to give us some feedback, or if you want to share any ideas for creating another asset.

Email: pixelnarval@gmail.com



Changelog

- Version 1.0 [August 26, 2023]:
  - Release version with 33 Prefabs.



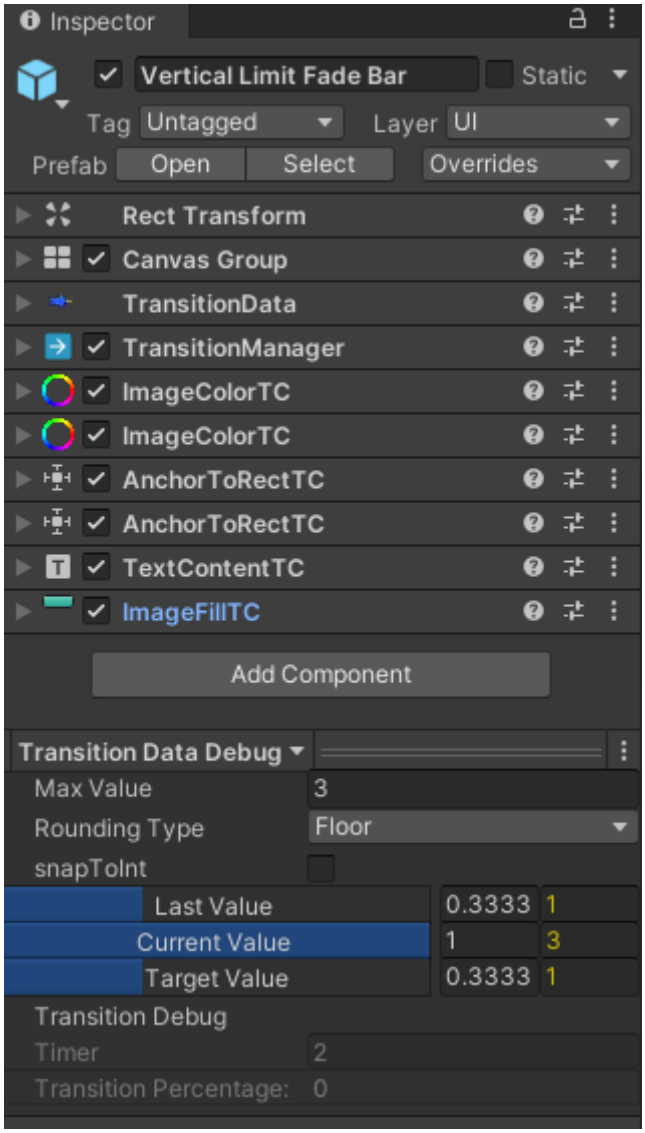
Setup

Introduction

All assets contain prefabs that can be used out of the box in a game project, they need some initial configuration and some other element to call their functions to work. There are also demo scenes showing different scenarios in which they can be used.

The easiest way to use any of our assets is just to drag a prefab into your game scene, set the size as you would with any other *UI* element, and customize the aspects by changing the fields in the transition components.

The prefabs are not only a good start but also a reference of what can be made, so the possibilities only depend on the creativity of the users. For those with more advanced knowledge, there's also the possibility to create a Health bar or any transition Object from scratch instead. The components are able to handle any custom-made *GameObject* structure, the user just has to set the proper components to add movement, color, and any other custom effects.

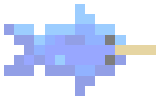


Components

A health bar, a progress indicator, and a loading screen are essentially visual representations of a transition between numeric values. That's why we decided to separate the different parts of a transition, separating the data, the logic, and the aesthetics the way it's usually done in software projects (A Model View Controller architecture).

All those parts are *Unity Components* that live inside the same *GameObject* (or distributed among its children). There are three kinds of components:

- The data is represented by a single component called ***TransitionData***. This component stores the current state of the transition (heath bar, loading screen, etc.) and sends this data to every other component in the *GameObject*.
- The way the transition changes its value to another is controlled by a component called ***TransitionUpdateManager***. A transition update manager allows defining the duration, speed, and



many other factors to determine the way the current value changes. As the TransitionData, there should only be one *TransitionManager* in the same *GameObject*.

- Lastly, the *Transition Components* are the visual controllers of a transition. There are many of these kinds of components, each one with a specific function inside the *GameObject*. Some of them control the Texts, the scales, movement, visual effects, and many more. There can be any number of *Transition Components* in the same GameObject depending on the level of *customization* the user wants.

  Note: In the prefabs provided, almost all *Transition Components* live in the root of the *base GameObject's* inspector window because it's easier to see all of them. But they can also be added to the children's objects for legibility.

Add Components

The easiest way to add new components is by using the "Add Component" button below the *GameObject* and searching inside **PixelNarval/Transition/** or using the menus at the top of the screen **Component -> PixelNarval/Transition/**.

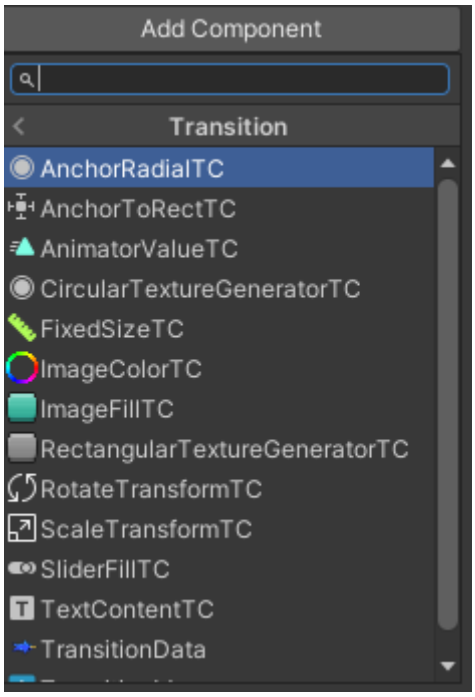


Figure 1: (Adding components using the Add Component Button on the GameObject)

Note: Not all the components listed in the image are present in all plugins. Each plugin comes only with the necessary components for it to work, this is a general guide.

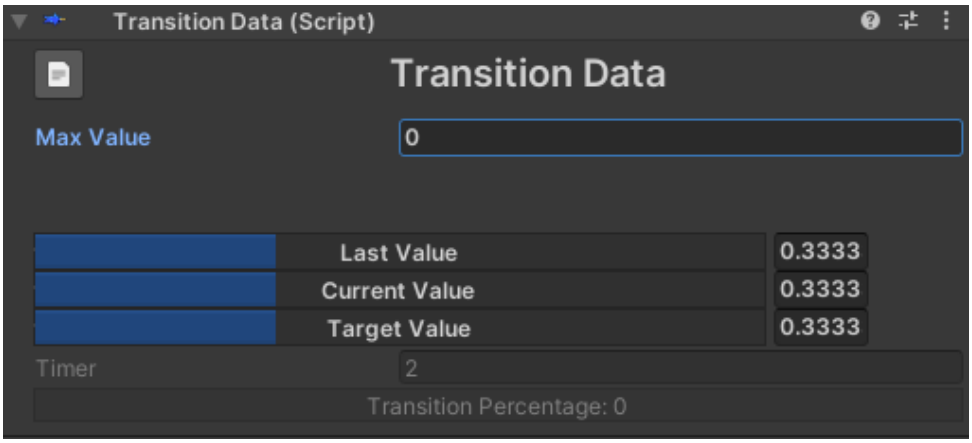
Transition Components

TransitionData

A *TransitionData* is the main component of this plugin. It contains the status information and allows the configuration of the main variables such as the current value, the max value, and many more.

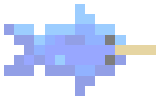
This component is also a great way to test the asset configuration without having to be in Play mode. Changing the current value slider will update all the *Transition Components* in editor time. So a good practice is to test everything on the editor or even as a prefab before clicking Play.

Note: The custom inspector allows setting the initial values for the transition but changing these inspector values at run time will also trigger changes on the interface. especially the last and target values because they will make the asset start a transition.



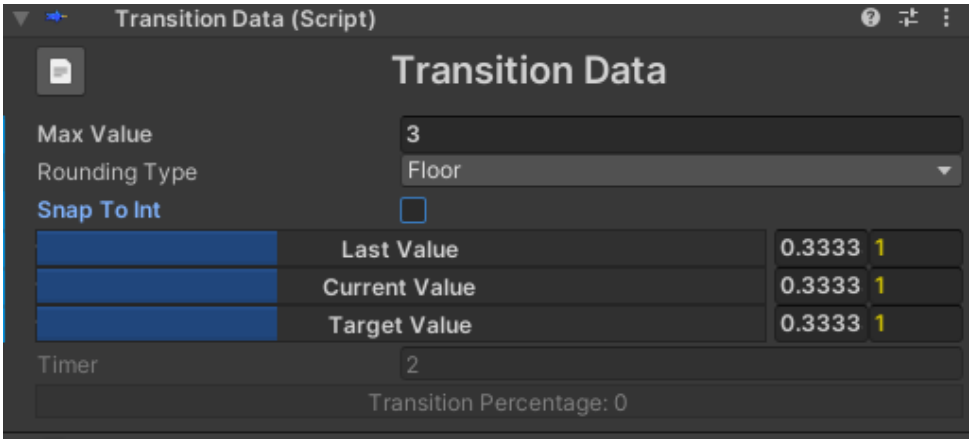
Inspector:

- **Documentation Button**: The button on the header opens the general component Documentation (this one).
- **Max Number**: Allows setting whole numbers in the transition instead of the default float representation (a decimal number between 0 and 1. This also sets the Maximum int value the transition can have when is full. Setting a Max Value to a number greater than 0 will display additional settings in the inspector.
- **Values**: There are three main values for each transition. Each has a slider that can be set to visualize the values, but can also be set by typing on the box on the right side.
  - **Last value**: is used mostly internally to set the value the transition had just before starting a transition. If the user sets this value in the Unity editor and then starts the game, the transition current value will transit from this value to the target Value.
  - **Current Value**: This is the current value of the transition. Is the value that most of the components use to show information in the scene.
  - **Target Value**: This represents the value the transition is aiming for during a transition.

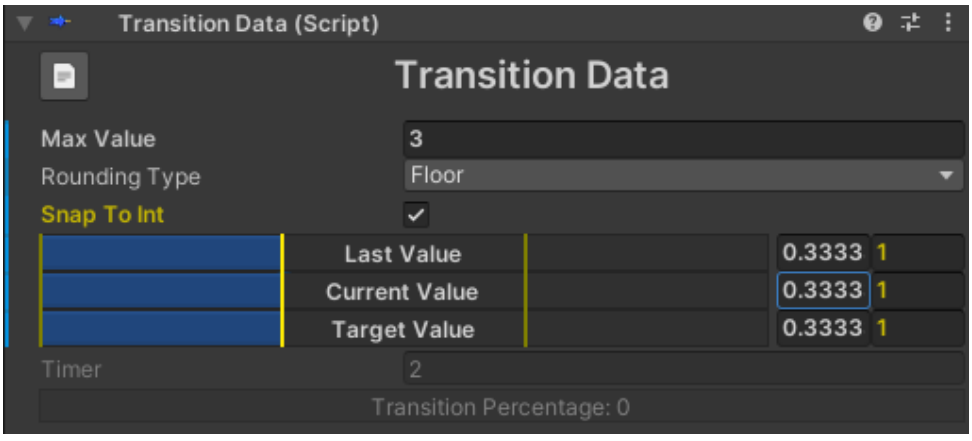


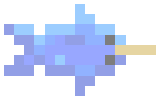
Setting this value will make the current value increase or decrease until it reaches this number.

Rounding Type: Once a Max Number is set, this field allows setting the rounding type for the float values into integers. The transition operates internally with float numbers so this will only affect how the asset looks and how you can use whole numbers instead of floats. Having a transition representing numbers allows representing the current state not only as a percentage but as a number and to perform operations such as adding some health or dealing a specific amount of damage in the case of life bars.



Having a max number and checking the Snap To Int toggle will enable visualizing integer snap points in the three transition values. This will help to view and set specific integer values and test how the rounding type would work. The segments have two kinds of graphics, the yellow ones represent the numbers the current value covers, and the pale yellow ones the previous and next integer values.





Functions And Properties

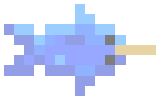
Changing values on the inspector is very useful. Still, the correct way to change values in-game is by calling some public functions for getting, setting values, or starting transitions from one value to another. This will be explained later in this section. Almost all the transition data is stored inside the class as public variables but some specific functionalities should be triggered through some functions and properties.

- **MaxNumber** (property): get and set the value for Max number.
- **SetMaxNumber (int value)**: Set the Max Number. This function Will keep the other values so the relative value will be kept.
- **SetMaxNumberOnly (int value)**: Set the Max Number but keep the current Int value;

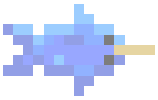
lastValue, currentValue, targetValue (variables): These variables can be used to change the base values of the transition. There are several properties that allow to change the values in different ways:

- **FloatValue**: Represents the current float value (from 0 to 1).
- **IntValue**: The integer number calculated by converting float value relative to its current max value. Use this value to set the current amount of health a character has.
- **AdjustedFloatValue**: The float value result of the rounded integer equivalent value. This property should only be used if the shared Max Number is greater than 0.
- **Percentage**: When a transition from the last value to the target value occurs, this value shows the current percentage amount until completion. This value will not be modified when testing on the editor.
- **Timer**: Similarly to Percentage, this value will only show a value other than 0 when a transition is active and will show the time until the transition is over.
- **StartTransitionTo (float value, System.Action<TransitionEventValue> onTransitionEnd = null)**: This is the main function to use for changing the transition value. This starts a transition between the current value and the specified new target value. There's an analog function that receives an int value. onTransitionEnd is a callback that will be called once the current value reaches the current target value. Be careful not to start a new transition before the values reach the target because the new transition will replace the previous and the same for the callback. There's also another variant for this function that has a parameter called "fromValue", This overload sets the last value of the *TransitionData* to this value before starting the transition, so the transition will not start from the current value but the given value.
- **ForceCurrentValue (float value, bool immediate)**: Allows skipping the transition and directly setting the last, current, and target value to the new one.
- **AddIntValue(int value)**: Adds (or subtracts if negative) an integer value from the current int value.

There are also two system events in *DataManager*:

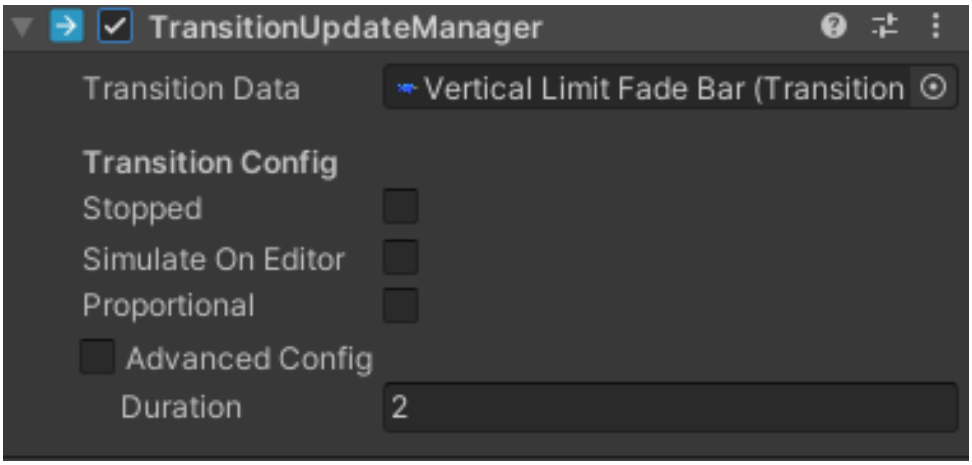


- **TransitionStartEvent**: Subscribe to this event to know when a transition starts.
- **TransitionEndEvent**: This event triggers once a transition ends.



TransitionUpdateManager

This component is required for a TransitionData to change from one state to another. It contains a lot of possible customizable fields to define how the current value should approach the target value.

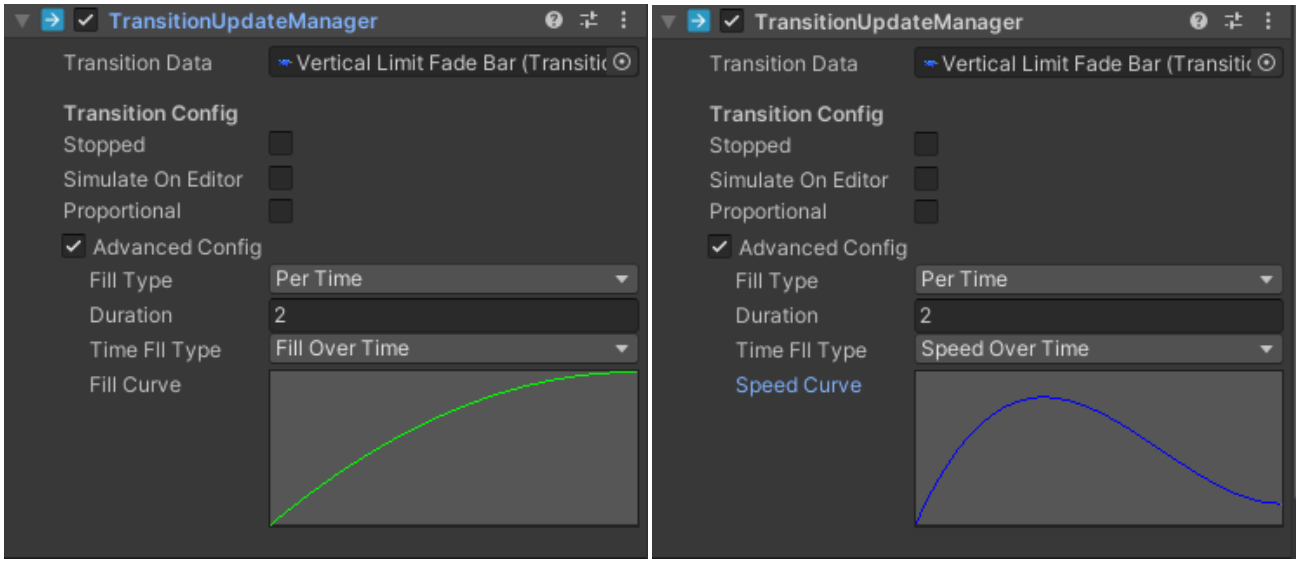


Inspector:

- **Stopped**: Use this field to prevent the transition from updating. It works the same as disabling the *Component*.
- **Simulate On Editor**: Having this field on will simulate updates on the transition while not in playtime. It is not precise so it should only be used as a preview.
- **Proportional Duration**: If set, the duration of a transition will be scaled depending on the variation between the target value and the last value making the duration for each transition not constant but variable depending on the changing amount. For example, having a reduction of 50% (from 100 HP to 50 HP) of the value will reduce the duration of the current transition to half the total set in the field Duration.
- **Advanced Config**: This toggle allows choosing between a simple or a more advanced configuration for the way the transition should be made. This will show additional settings in the inspector.

The simple mode will generate a transition with a constant speed.

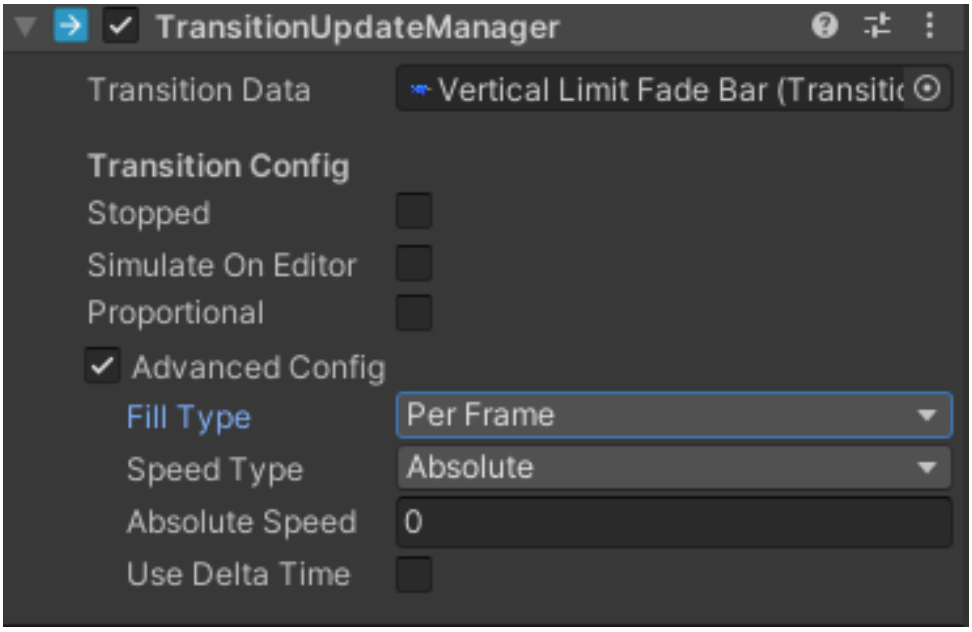
- **Duration**: Determines the total time a transition would take to complete.



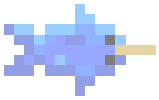
The advanced mode allows more control of the behavior of the transition in exchange for a more complex set of settings. It is only recommended for advanced users.

Fill Type: There are two types of advanced settings, each one with its own configuration fields.

- **Per time**: This kind of transition is based on a time duration.
  - **Time Fill Type**: sets which value will change over the time duration.
    * **Fill Over Time**: Will set the current value of the transition depending on how much time has passed. **Warning**: The curve should end in a Y value of 1 or the transition will not finish!
    * **Speed Over Time**: This setting allows the speed to change over the time duration. **Warning**: The curve should have at least one segment greater than 0 to work.



- **Per Frame**: This configuration will make the transition value advance at a constant rate for each frame by adding or subtracting a fixed amount for each frame.
- Speed Type: This field allows choosing between increases of an absolute value or a percentage.
  - **Absolute**: uses a fixed amount based on the Max Value. **Warning**: A Max Value greater than 0 is needed for this option to work.
  - **Percentage**: fills a fixed percentage of the current value every frame.
- **UseDeltaTime**: Marking this as true will multiply the added amount by Time.Deltatime. This will make the amount related to the time between updates of the project, so the transition will maintain a relatively constant speed even with a variable framerate.

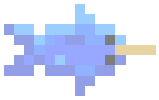


Common Fields

All *Transition Components* contain an **Order** field. It's used to determine the order in which all components are executed. This is important because some components need another to run first for them to work correctly. For example, you can add a component to set a color gradient for the fill of a life bar and then another that shows a flash once a transition starts. The second one modifies the color of the first, so the user should make sure the first one has a lower order value, or unwanted behaviors may occur.

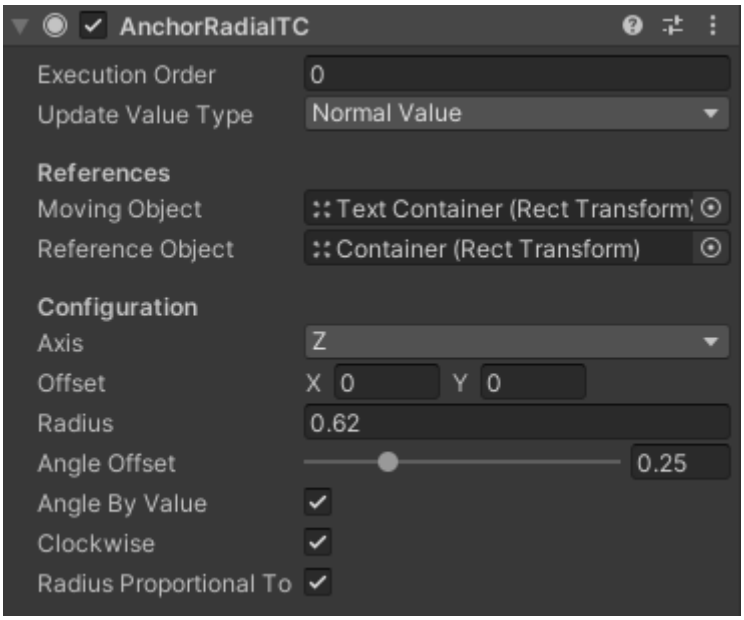
There's also another common field named **Update Value Type** which defines the current value used in the script using the float representation or the adjusted equivalent value. This means that the value will increase as steps depend on the max number. Use this to represent non-continuous transitions of color gradients, texts, etc.

The rest of the fields are unique for every component. Here's a list of all the existent Transition Components:



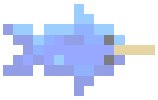
AnchorRadialTC

This component binds the position of a *RectTransform* to another using its position as a reference and following a radial trajectory around it. It makes a *Transform* move around another without changing its rotation.



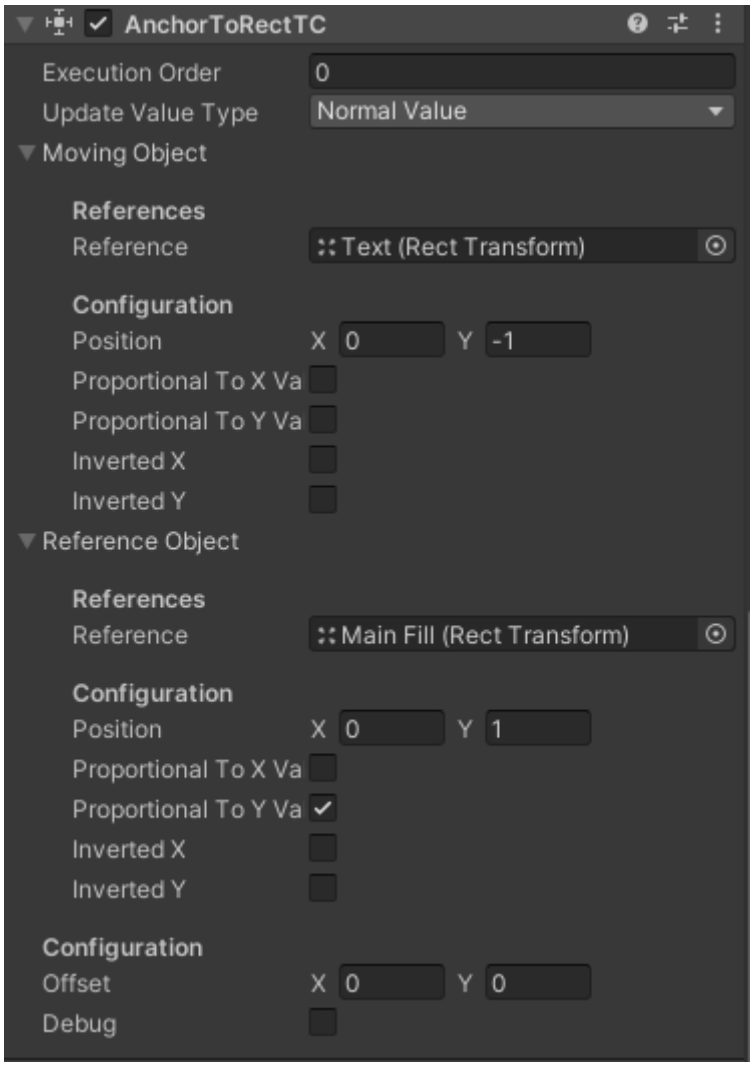
Inspector:

- **Moving Object**: The object that will be moved.
- **Reference Object**: The referenced object that will be used to calculate the position to place the Moving Object.
- **Axis**: the axis (X, Y, or Z) that defines the rotation around the Target Object. For most *RectTransforms*, this value should be Z.
- **Offset**: Additional spacing to add.
- **Radius:** The Distance from the Reference Object.
- **Angle Offset:** Additional angle added so the 0 angle is tilted.
- **Angle By Value:** When this is on, the angle value is given by the *TransitionData* current value.
- **Clockwise:** Should the angle increase the way the clock spins?
- **Radius Proportional To Rect Width:** Makes the distance proportional to the Rectangle dimension X.



AnchorToRectTC

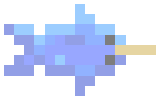
This component binds the position of a *RectTransform* to another using its position as a reference and following. It makes a Transform move. Text components for example are usually bound to parts of the *GameObject* so they move along the axes.



Inspector:

- **Moving Object**: The object that will be moved.
- **Reference Object**: The referenced object that will be used to calculate the position to place the Moving Object.
- **Offset**: Additional distance between objects.
- **Debug**: Turn on to Debug some lines showing the target position.

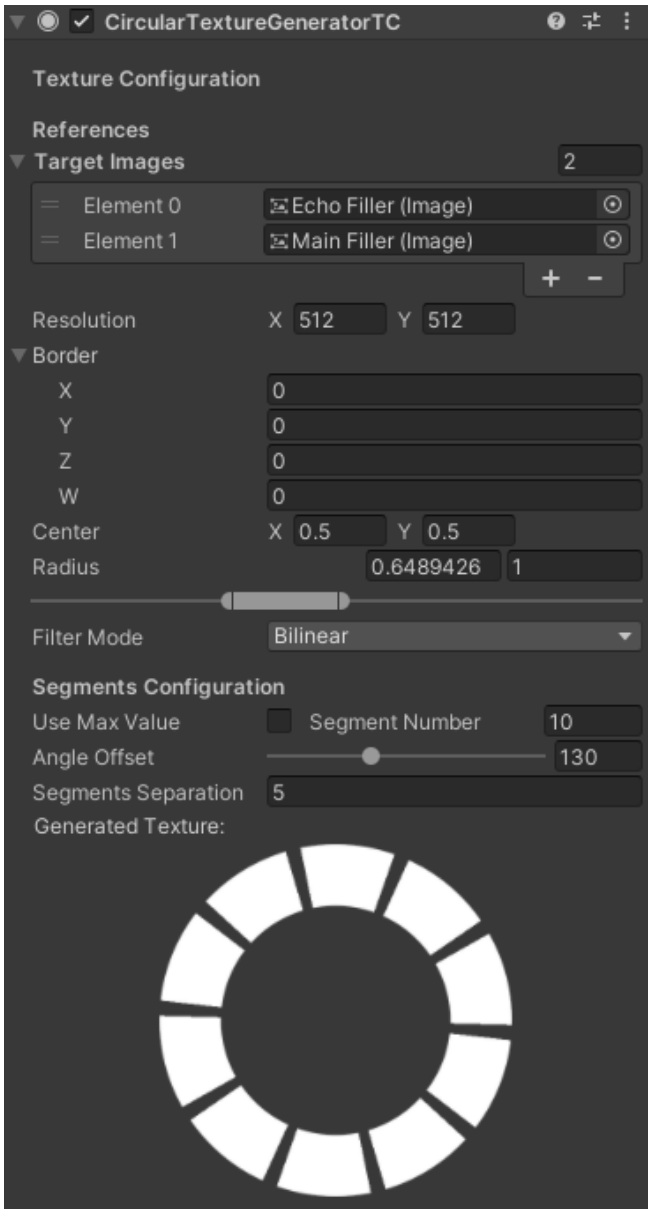
Both Reference Objects contain the same fields:



- **Reference:** The reference to the *RectTransform*.
- **Position**: Vector2 with the relative anchor point to the Rectangle with 0, 0 as the center. A 1 value means the border of the rectangle in the positive axis, and a -1 is the negative border.
- **Proportional To Value Horizontal**: Uses the current value in the *TransitionData* instead of the Position X value.
- **Proportional To Value Vertical**: Uses the current value in the *TransitionData* instead of the Position Y value.
- **Inverted Horizontal Direction**: Inverts the X position. This is used when using the current value to reverse the motion of the Moving Object.
- **Inverted Horizontal Direction**: Inverts the Y position. This is used when using the current value to reverse the motion of the Moving Object.

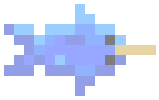
CircularTextureGeneratorTC

This component creates a circular *2DTexture* and assigns its value as a sprite to a target Image component. Its purpose is to generate custom circular-based textures to be used in bar transitions and avoid having to create each texture and import it into the project.



Inspector:

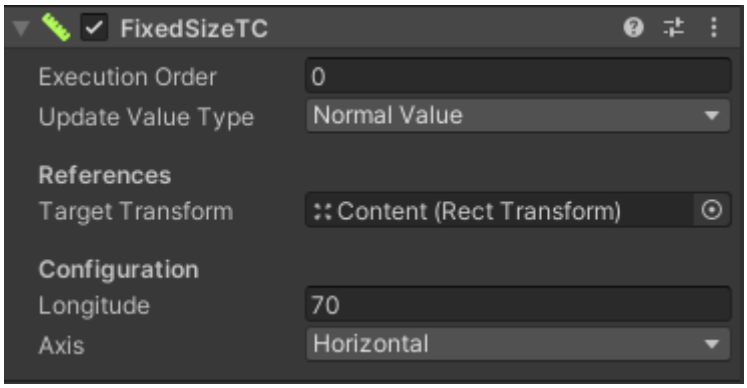
- **Target Images**: The list of *Images* to which the generated textures would be assigned.
- **Resolution**: The resolution of the texture to generate.
- **Center**: The center of the drawn circle. Move it to the borders or the corners to create semi-circles.



- **Radius**: The distance from the center to the outer and inner parts of the circle (or ring).
- **Filter Mode**: Changes the image filter mode. Change it to modify the interpolation algorithm so the pixel images get smoother or sharper.
- **Color**: The default color for the circle pixels. The default color is white so it can be masked using the *Image* color field.
- **Transparent Color**: The color for the pixels outside the circle and between the segments. The default color is white with no alpha (transparent).
- **Use Max Value**: Turning this on makes the number of segments to be determined by the *TransitionData*'s Max Value.
- **Segment Number**: Number of angular segments the circle is made of. This will cause the circle to become segmented into radial pieces.
- **Angle Offset**: Changes the starting position for the angle 0 of the circle. Use this for adjusting the rotation of the segments.
- **Parts Separation Angle**: The distance in angles between the segments.

FixedSizeTC

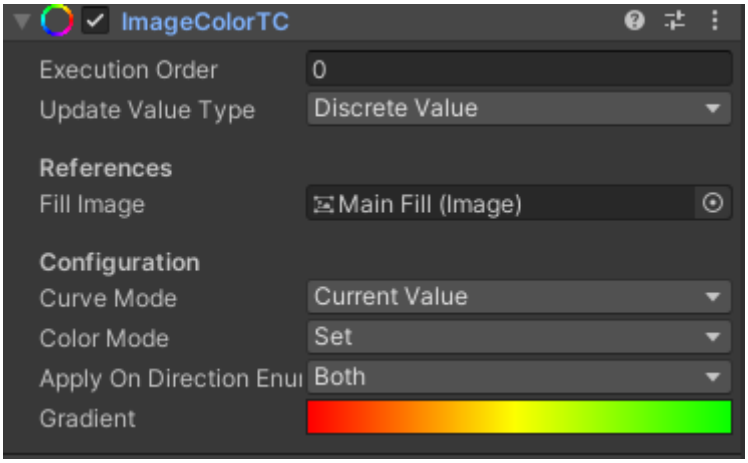
This util TransitionComponent is a quick way to set the size in one axis for a *RectTransform*. Is mostly used to increase or decrease the length of a health bar.



Inspector:

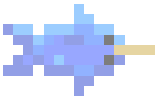
- **Target Transform**: the transform that will change size.
- **Longitude**: new size among axis.
- **Axis**: the axis (horizontal or vertical).

ImageColorGradientTC



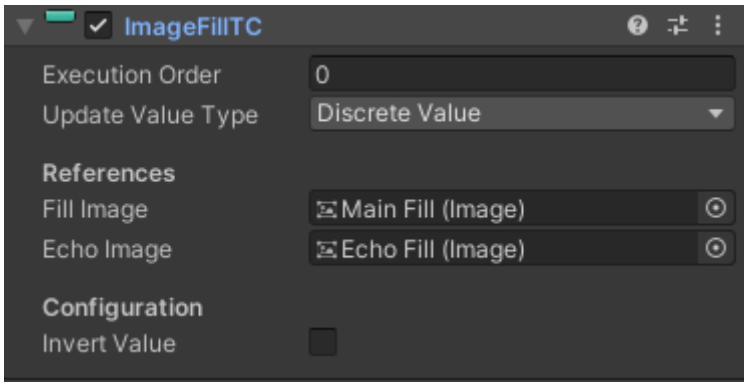
Inspector:

- **Fill Image**: the target image (*UnityEngine.UI.Image*).
- **Curve Mode**: The color may depend on two values: the current value or the percentage of the transition. The second one makes it possible to change the colors each time a transition occurs, for example, adding a red color each time a health bar decreases in value.
- **Color Mode**: This allows the application of color operations between the current color of the image and the color according to the color gradient in this component.
- **Applied On Direction**: The color will be applied only when the current value has been increased, decreased, or both.
- **Gradient**: The color to be applied according to the current amount in the TransitionData or the time of the current transition.



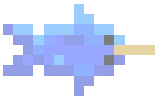
ImageFillTC

This component Allows changing the Fill Amount value of an *Image* component according to the *TransitionData* current and target values. It makes the image fill while the transition value increases and the opposite as the value decreases. The image should have a Filled ImageType.



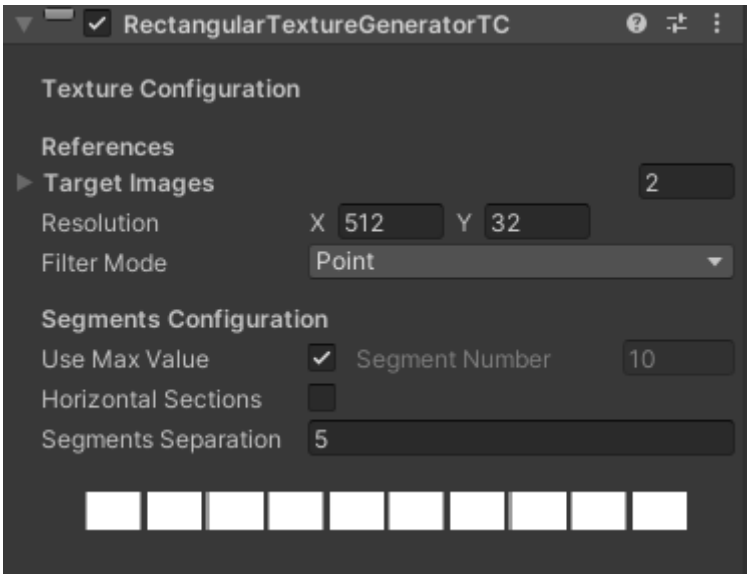
Inspector:

- **Fill Image**: Reference to the main *Image*.
- **Echo Image**: Reference to the secondary *Image* used to project the increase or decrease of the current value.
- **Invert Value**: Inverts the value obtained from *TransitionData* applying *1- the current value*. This is used to create an inverse transition where 0 means a full image and 1 means an empty one.



RectangularTextureGeneratorTC

This component creates a rectangular *2DTexture* and assigns its value as a sprite to a target *Image* com- ponent. Its purpose is to generate custom rectangular-based textures to be used in bar transitions and avoid having to create each texture and import it into the project

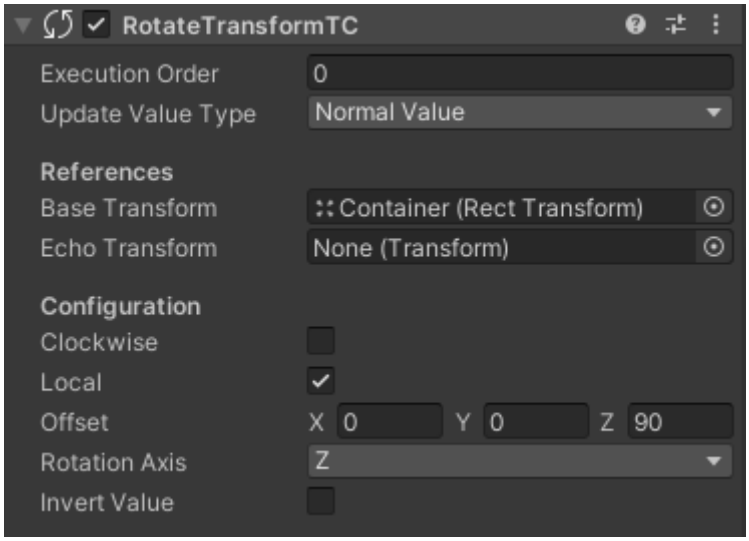


Inspector:

- **Targets**: The list of images to which the generated textures would be assigned.
- **Resolution**: The resolution of the texture to generate.
- **Filter Mode**: Changes the image filter mode. Change it to modify the interpolation algorithm so the pixel images get smoother or sharper.
- **Color**: The default color for the circle pixels. The default color is white so it can be masked using the *Image* color field.
- **Transparent Color**: The color for the pixels outside the circle and between the segments. The default color is white with no alpha (transparent).
- **Use Max Value**: Turning this on makes the number of segments to be determined by the *TransitionData*'s Max Value.
- **Segment Number**: Number of angular segments the circle is made of. This will cause the circle to become segmented into radial pieces.
- **Angle Offset**: Changes the starting position for the angle 0 of the circle. Use this for adjusting the rotation of the segments.
- **Parts Separation Angle**: The distance in angles between the segments.

RotateTransformTC

This component changes the rotation of the target transform depending on the current value of the *TransitionData*.

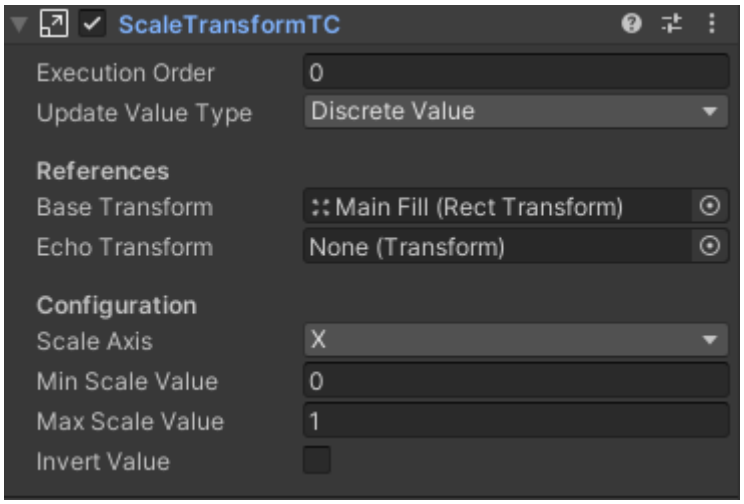


Inspector:

- **Base Transform**: Reference to the main *Image*.
- **Echo Transform**: Reference to the secondary *Image* used to project the increase or decrease of the current value.
- **Clockwise**: The rotation should rotate to the right as the clock spins?
- **Local**: Should rotate locally or in world space.
- **Offset**: Additional angle to the rotation so the angle 0 is tilted.
- **Rotation Axis**: The axis to rotate around.
- **Invert values**: Inverts the value obtained from *TransitionData* applying *1- the current value*. This is used to create an inverse transition where 0 means a full image and 1 means an empty one.

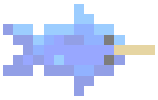
ScaleTransformTC

This component changes the scale of the target *Transform* depending on the current value of the *TransitionData*. The Scale would change between 0 and 1



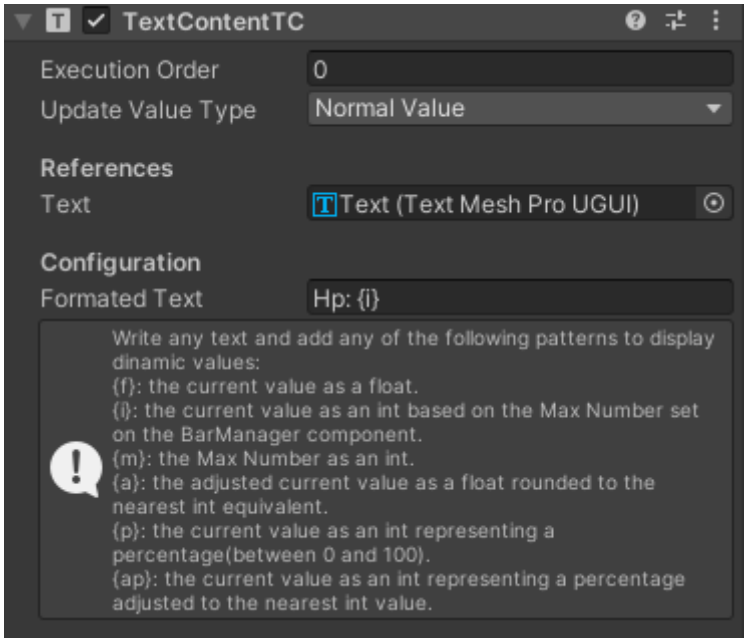
Inspector:

- **Base Transform**: Reference to the main *Image*.
- **Echo Transform**: Reference to the secondary *Image* used to project the increase or decrease of the current value.
- **Scale Axis**: The axes among the *Transform* would be escalated.
- **Min Scale Value**: The minimum value the scale can be.
- **Max Scale Value**: The maximum value the scale can be.
- **Invert values**: Inverts the value obtained from *TransitionData* applying *1- the current value*. This is used to create an inverse transition where 0 means a full image and 1 means an empty one.



TextContentTC

Changes the string of a *TextMeshPro Text* component using the values on the *TransitionData*.



Inspector:

- **Text**: the reference for the *TextMeshPro Text* component.
- **Formatted Text**: The text to be added to the component. It uses some special patterns that are replaced by their corresponding Transition values.

These are the patterns:

- **f**: The current value as a float.
- **i**: The current value as an int based on the Max Value set on the *TransitionData* component.
- **m**: The Max Value as an int.
- **a**: The adjusted current value as a float rounded to the nearest int equivalent.
- **p**: The current value as an int representing a percentage (between 0 and 100).
- **ap:** The adjusted current value as a percentage. A value of 3/ 5 will show 60.